

Advancing 2D LiDAR-based Indoor Localization using Line Normal-constrained Particle Filters

Waseem Mohammed

Abstract—Particle filters are a fundamental tool for robot localization, capable of estimating a robot’s position and orientation even in the presence of multimodal uncertainty distributions. While robust, conventional particle filters face significant challenges, such as particle degeneracy and the trade-off between accuracy and robustness, particularly when dealing with map discrepancies. The observation model’s improvements can enhance robustness but risk reducing its informativeness, potentially reducing its overall accuracy.

This research proposes a method to improve localization accuracy and reliability in 2D environments by incorporating line normals from 2D LiDAR data into the particle filter framework. Inspired by the successful application of surface normals in 3D localization systems by Heruka Andradi et al [1], this research adapts these principles for 2D environments to enhance localization precision and reliability. The proposed methodology is rigorously benchmarked and tested against standard particle filters to evaluate its benefits.

The results aim to demonstrate substantial improvements in accuracy and performance, advancing the state of indoor localization by optimizing particle filter processes for 2D environments.

Index Terms—Particle Filters, 2D LiDAR Localization, , Line Normals

I. INTRODUCTION

Particle filters represent a vital tool in the domain of localization, particularly in indoor environments. Effective localization enables robots to determine their position and orientation within a given space, enabling functionalities like navigation, planning the path, and avoiding collisions.. In these filters, the robot’s pose belief is modeled as a set of weighted samples, updated recursively through Bayesian filtering. Unlike Kalman filters, which are limited to unimodal Gaussian distributions, particle filters can handle arbitrary multimodal uncertainty distributions, making them particularly suitable for complex and dynamic environments [2]. LiDAR-based localization systems, commonly used in particle filters, are favored for their high depth accuracy, insensitivity to lighting conditions, and independence from external infrastructure, making them particularly suitable for autonomous navigation in indoor environments.

However, localization in indoor environments poses significant challenges. Ambiguities in environmental features, sensor noise, and discrepancies between the map and the real-world environment make it difficult to achieve robust and precise localization. Particle filters are commonly employed

to address these challenges. Despite their strengths, particle filters suffer from issues such as particle degeneracy and the trade-off between robustness and accuracy when dealing with map discrepancies.

This research addresses these challenges by extending the principles of surface normal-based localization from 3D to 2D, providing a cost-effective and robust solution for indoor environments using 2D LiDARs.

A. Motivation

Addressing these challenges has inspired innovative approaches, including integrating surface normals derived from 3D LiDAR data into the particle filter framework. This method enhances the filter’s ability to balance accuracy and robustness, achieving significant improvements in indoor localization [1]. However, 3D LiDAR systems have notable limitations, such as high cost, limited range, and constrained field of view, which make them impractical for widespread use.

In contrast, 2D LiDARs are more affordable, readily available, and extensively used in robotics. This research builds upon the success of surface-normal-based 3D localization by proposing to adapt the methodology to 2D environments using 2D LiDAR. Extending the surface normal-based localization approach to 2D LiDAR systems can significantly enhance the accessibility and applicability of the algorithm. By leveraging line normals extracted from 2D LiDAR data, the proposed framework aims to provide a cost-effective, reliable, and accurate localization solution. This research aims to achieve comparable performance to the 3D algorithm while addressing the challenges associated with conventional particle filters.

B. Problem Statement

Standard particle filters used in LiDAR-based localization face two major issues:

- 1) **Particle Degeneracy:** When the robot’s motion model produces noisy predictions, the majority of particles receive negligible weights based on the observation model, resulting in a failure to adequately represent the true system state. This results in inefficient sampling and reduced localization accuracy.
- 2) **Map Discrepancies:** In real-world scenarios, maps may not perfectly match the operating environment due to mapping errors, changes, or incompleteness. This discrepancy often leads to a trade-off between accuracy and robustness. Attempts to improve robustness by modifying the observation model may inadvertently reduce the model’s informativeness, resulting in a decline in accuracy [3] [4].

*Submitted to the Department of Computer Science at Hochschule Bonn-Rhein-Sieg in partial fulfilment of the requirements for the degree of Master of Science in Autonomous Systems

[†]Supervised by Prof. Dr Paul G. Plöger and M.Sc. Deebul Sivarajan Nair

[‡]Submitted in January 2025

Moreover, addressing the robot kidnap problem, where the robot's estimated pose is severely disrupted due to sudden repositioning remains a significant challenge for standard localization methods.

C. Proposed Approach

This research proposes adapting the surface normal-based 3D localization algorithm to 2D by incorporating line normals from 2D LiDAR scans into the particle filter framework. The key contributions include:

- 1) **Incorporation of Line Normals:** The algorithm utilizes line normal information derived from 2D LiDAR data, improving the informativeness of the observation model without compromising robustness.
- 2) **Efficient Particle Filter Implementation:** The proposed approach mitigates particle degeneracy by incorporating line normal information, thereby enhancing both the filter's efficiency and its accuracy in maintaining a consistent representation of the robot's pose.
- 3) **Handling the Robot Kidnap Problem:** The algorithm is designed to recover effectively from significant pose misalignments, demonstrating resilience in challenging scenarios.

The proposed methodology is benchmarked against Adaptive Monte Carlo Localization (AMCL), a standard in 2D localization, to evaluate its performance in terms of accuracy, robustness, and recovery capabilities. This research bridges the gap between 3D and 2D localization techniques, offering a practical and reliable solution for autonomous navigation in dynamic indoor environments.

II. RELATED WORK

The field of localization using particle filters has seen significant advancements to address challenges such as particle degeneracy, map discrepancies, and computational efficiency. Among these, the optimal proposal distribution has been explored as a promising solution to minimize particle degeneracy by aligning particle weights more effectively with observations. While the theoretically optimal proposal distribution is challenging to compute in practice, approximations have been developed to achieve feasible implementations.

Grisetti et al. [5] introduced a Gaussian approximation of the optimal proposal distribution through scan matching techniques. Although effective, this approach involves multiple sampling steps and repeated evaluations of observation likelihoods, resulting in considerable computational overhead. Blanco et al. [6] proposed an alternative that bypasses scan matching, focusing instead on a non-parametric approach to reduce computational demands. However, both methods face limitations in terms of efficiency, especially for LiDAR-based localization, where observation likelihoods are complex and non-parametric.

Handling map discrepancies arising from dynamic obstacles or mismatches between the map and environment has also been a key focus in the literature. Jianchao et al. [7] introduced a motion detection module to filter out dynamic objects, while approaches such as temporary maps [8] [9] addressed

semi-static objects by maintaining short-term environment representations. Tipaldi et al. [10] extended this idea further by incorporating environment estimation into multi-session SLAM. However, these methods typically depend on accurate localization estimates and can impose significant computational burdens.

The foundation of this research lies in the surface normal-based localization framework introduced for 3D environments by Andradi et al. [1]. The original work utilized surface normals extracted from 3D LiDAR scans and maps to approximate the optimal proposal distribution through a closed-form Gaussian representation. This approach demonstrated substantial improvements in accuracy and robustness while mitigating particle degeneracy. Additionally, it introduced a dual-model observation strategy, employing separate models during particle propagation and weight update steps to address map discrepancies effectively.

In the present study, the focus is on adapting this 3D localization algorithm to 2D LiDAR-based environments. The methodology directly builds upon the 3D algorithm outlined in [1], applying the same principles to derive line normals from 2D LiDAR data and incorporating them into the particle filter framework. While the underlying algorithm remains unchanged, the adaptation to 2D involves leveraging 2D line features instead of 3D surface normals, aligning with the constraints and opportunities of 2D LiDAR systems.

This work differs from other related studies in that it directly translates a proven 3D algorithm to 2D without introducing additional modifications or alternative techniques. The goal is to evaluate how effectively the principles of surface-normal-based localization can enhance performance in 2D environments, maintaining the computational efficiency and robustness achieved in the original 3D implementation.

III. METHODOLOGY

Building upon the principles of optimal 3D localization, this project adapts and implements these concepts within a 2D LiDAR-based framework to develop an accurate and robust particle filter-based localization system for indoor mobile robots. The primary aim is to enhance the accuracy and robustness of LiDAR-based indoor localization by addressing two critical challenges: particle degeneracy and the trade-off between accuracy and robustness caused by map discrepancies.

The particle degeneracy problem is tackled through an efficient optimal particle filter that incorporates constraints derived from line normals in both the map and LiDAR scans. The accuracy-robustness trade-off is addressed by employing two distinct observation models: a high-fidelity model for precise particle propagation and a low-fidelity model to ensure robust weight updates in environments with map discrepancies. This represents a direct adaptation of the principles successfully applied in 3D surface normal localization, with adjustments to compute and utilize line normals specific to 2D data.

To assess the effectiveness of the proposed approach, a comparative analysis is conducted against a baseline system utilizing the Adaptive Monte Carlo Localization (AMCL)

package of ROS Noetic, which is widely used for standard particle filter-based localization. Evaluations are performed in a simulated environment, while the system is implemented in a real-world setup without formal evaluation. Since the system relies on a map containing line normal information, the methodology includes a brief explanation of the line normal extraction process. Unlike surface normal extraction in 3D systems, line normals in 2D are derived from map and scan data, with specific details provided in a dedicated subsection.

The methodology leverages these approaches to significantly improve localization accuracy and robustness in indoor environments. Further details of the baseline system, the proposed method, and the evaluation process, including datasets and performance metrics, are provided in subsequent sections. These contributions collectively establish a robust and dependable 2D localization framework tailored for indoor environments.

A. Overview of Standard Particle Filter

The particle filter is a recursive Bayesian approach designed to estimate the state of a dynamic system, such as the position and orientation of a robot. It captures the robot's pose belief through a collection of particles, each assigned a specific weight, called particles. These particles are propagated and updated based on sensor measurements, which allows the filter to handle non-linearities and multimodal distributions, making it ideal for robot localization.

The standard particle filter functions through three core phases: Prediction, Correction, and Resampling.

- 1) Prediction Step: During this phase, the particles are propagated forward in time based on the robot's motion model. The state of each particle at time t is predicted from its previous state $x_{t-1}^{(i)}$ and a control input u_t , which represents the robot's motion (e.g., velocity or steering commands). The prediction also includes a noise term to account for motion uncertainty. The predicted state of each particle $x_t^{(i)}$ is given by:

$$x_t^{(i)} = f(x_{t-1}^{(i)}, u_t) + \epsilon_t$$

Where:

- $x_t^{(i)}$ is the predicted state of particle i at time t ,
- $f(x_{t-1}^{(i)}, u_t)$ is the motion model that predicts the state based on the previous state $x_{t-1}^{(i)}$ and the control input u_t
- ϵ_t represents the noise term to account for uncertainties.

This step accounts for the robot's motion uncertainty and allows the filter to propagate particles based on the latest control inputs.

- 2) Correction Step: After prediction, the weights of the particles are adjusted according to the likelihood of the observed sensor measurement z_t relative to the predicted states of the particles. This step refines the predicted particle set by integrating the observation model, which evaluates the predicted sensor readings against the ac-

tual measurements. The weight of each particle $w_t^{(i)}$ is updated as

$$w_t^{(i)} = p(z_t | x_t^{(i)})$$

Where:

- $w_t^{(i)}$ is the weight of particle i at time t ,
- $p(z_t | x_t^{(i)})$ is the likelihood of the sensor measurement z_t given the predicted state $x_t^{(i)}$,

The correction step adjusts the particles to reflect the likelihood that each predicted state matches the actual sensor observations.

- 3) Resampling Step: The final step addresses particle degeneracy, a situation where the majority of particles carry insignificant weights, leading to poor state estimation. Resampling is performed to focus on the particles with higher weights, ensuring that the particle set remains representative of the belief distribution. The new set of particles P' is generated by resampling particles based on their weights $w_t^{(i)}$, with particles that have higher weights being more likely to be chosen:

$$P' = \{x_t^{(i)} | w_t^{(i)} \text{ chosen with probability } w_t^{(i)}\}$$

Where:

- P' is the new set of particles after resampling,
- Particles are selected according to their weights, meaning particles with higher weights have a higher probability of being selected.

This step ensures that the particles representing the robot's state are properly distributed and reduces the problem of particle degeneracy.

The particle filter process, as described above, is widely used in robotics, particularly in localization tasks such as those found in mobile robots using LiDAR or other sensors. The method's ability to represent multimodal distributions and handle complex environments makes it an essential tool in modern localization systems [11]

B. Line Normal Extraction from 2D LiDAR Data

Normal constraints are a cornerstone of this research, inspired by the proven success of incorporating surface normals in 3D localization. However, extracting such normals from 2D LiDAR data presents unique challenges since surface normals inherently require 3D geometric surfaces, which are absent in 2D data. For 2D LiDAR scans, the data is limited to x and y coordinates with a constant z -axis, making traditional surface normal computation infeasible.

To adapt to this limitation, two approaches were implemented to extract meaningful geometric features from the 2D LiDAR data:

- 1) *Pseudo-3D Normal Estimation*: In this method, the 2D LiDAR data was transformed into a pseudo-3D format to facilitate normal estimation. The transformation was achieved by interpolating along the z -axis, where additional layers were generated by maintaining the original x and y coordinates while assigning incremental z -values. Using the laser geometry ROS package [12], the 2D scan was first converted

into a point cloud. Additional layers were then created by incrementally assigning new z -values while keeping the x and y coordinates constant, effectively forming a 3D structure:

$$z = k \cdot \Delta z, \quad k = 1, 2, \dots, N$$

where N is the number of interpolated layers, and Δz is a constant vertical offset.

Once the pseudo-3D point cloud was constructed, surface normals were calculated using the `pcl` library. Specifically, the `NormalEstimation` method was employed to compute the orientation of the local surface at each point. This approach determines the normal vectors by analyzing the spatial relationships between neighboring points within a specified search radius.

By leveraging these normal vectors, the transformed pseudo-3D data provided valuable geometric features that could be used in observation models, enhancing the overall robustness and precision of the localization system. This method effectively extended the utility of 3D surface normal constraints to the 2D LiDAR domain.

2) *Direct Line Normal Calculation*: For a more direct approach, line normals were computed from the 2D LiDAR points themselves. Since a single point has infinite potential normals, meaningful normals were extracted by first identifying line segments connecting consecutive points. The following steps were performed:

- 1) Point Segmentation: The raw scan data was segmented into a sequence of points (x_i, y_i)
- 2) Direction Vectors: For each point, direction vectors were calculated using its neighboring points:

$$v_{\text{prev}} = p_i - p_{i-1}, \quad v_{\text{next}} = p_{i+1} - p_i$$

Where $p_i = (x_i, y_i)$ is the point's position.

- 3) Normal Vector Calculation: The average direction vector was computed:

$$\mathbf{v}_{\text{avg}} = 0.5 \cdot (\mathbf{v}_{\text{prev}} + \mathbf{v}_{\text{next}})$$

A normal vector $\mathbf{n}_i$ perpendicular to $\mathbf{v}_{\text{avg}}$ was then calculated as:

$$\mathbf{n}_i = \begin{bmatrix} v_{\text{avg},y} \\ -v_{\text{avg},x} \end{bmatrix}$$

- 4) Orientation Adjustment: To ensure the normals pointed consistently toward the robot, the angle between the normal vector and the vector from the robot to the point was calculated. If the angle was greater than 90° , the normal's direction was inverted.

C. Map Generation

Map generation is a critical step in ensuring the success of the localization system. Accurate maps, enriched with geometric features such as line normals, form the foundation for reliable observation models and particle filter updates. This section discusses the methodologies employed for generating maps in both simulated and real-world environments, ensuring compatibility with the proposed line normal constraints and

observation models. The maps are generated using the Point Cloud Library (PCL) and saved in the `pcd` format. The saved `pcd` files contain both the point coordinates and the computed normals for every point, facilitating efficient use in the localization system.

1) *Simulation Map Generation*: The simulated map is generated by leveraging the Gazebo simulator to simulate LiDAR data from indoor environment models, following the approach described in [1]. Ground truth robot poses from the simulator are used to ensure accurate transformations during map generation. The methodology is as follows:

- Data Collection: The robot is driven around the simulated environment while collecting LiDAR scans.
- Normal Calculation: Surface normals for scan points are estimated using Principal Component Analysis (PCA) of neighboring points. This ensures reasonably accurate ($< 0.1\text{rad}$) normal estimates.
- Map Construction: The surface normal point clouds are transformed into the map frame and accumulated using an octree data structure.
- Downsampling: The accumulated point cloud is downsampled with a voxel size of 3 cm to optimize storage and computational efficiency.
- Storage: The final map, comprising both points and their corresponding normals, is stored in `pcd` format for use in the localization system.

This simulation map served as a controlled testing environment to evaluate the performance of the proposed localization approach in various scenarios, including occlusions, feature sparsity, and dynamic obstacles. This methodology is compatible with both the pseudo-3D normal estimation and direct line normal calculation methods, ensuring consistency and reliability across different approaches.

2) *Real-World Map Generation*: In this approach, the method for 3D surface normal localization referenced was based on LeGO-LOAM, which utilizes 3D LiDAR data to generate maps with surface normals for localization. However, since LeGO-LOAM requires 3D point cloud data as input and the real-world implementation in this work relied on 2D LiDAR data without interpolation, adapting LeGO-LOAM was deemed impractical. Instead, Gmapping was used to generate 2D maps, which were then converted into point clouds. Line normals were extracted from these point clouds, localized around the robot's position within a defined radius, and accumulated for analysis. This approach provided a practical solution for achieving normal-based localization in the absence of 3D LiDAR data.

- 1) Occupancy Grid Mapping: Gmapping was used to create a 2D occupancy grid map of the environment. This map, published on the `/map` topic, represented the environment in discrete grid cells where each cell's value indicated whether it was occupied, free, or unknown.
- 2) Point Cloud Extraction: The occupancy grid was converted to a 2D point cloud by extracting the grid cells marked as occupied. The transformation from grid coordinates (i, j) to world coordinates (x, y) was achieved using:

$$x = i \cdot r + x_0, \quad y = j \cdot r + y_0$$

where r is the map resolution, and (x_0, y_0) is the origin of the map.

- 3) Normal Calculation: Normals were estimated for each occupied point by analyzing the gradient of the occupancy values in the grid. Let ∇x and ∇y represent the changes in occupancy values along the x and y axes, respectively. The normal vector $\mathbf{n}$ was computed as:

$$\mathbf{n} = \frac{\nabla f}{\|\nabla f\|} = \frac{(\nabla x, \nabla y)}{\sqrt{\nabla x^2 + \nabla y^2}}$$

Here, f is the occupancy grid, and $\|\nabla f\|$ ensures normalization. The z -component of the normals was set to zero, as the data was 2D.

- 4) Point Cloud Refinement: To focus the mapping on relevant areas, only points within a specific radius of the robot were considered. This radius was chosen to balance computational efficiency and the density of features available for localization. Additionally, the normals were adjusted to point inward toward the robot's position by ensuring alignment with the vector from the point to the robot.
- 5) Curvature Calculation: The curvature κ at each point was approximated as the magnitude of the gradient:

$$\kappa = \sqrt{\nabla x^2 + \nabla y^2}$$

This value provides insight into the local shape of the surface represented by the point cloud.

- 6) Map Output: The final 2D point cloud, augmented with points, normals and curvature values, was converted to the .pcd format. This point cloud, containing features essential for localization, served as the input map for subsequent evaluations.

The approach offers several key advantages, including adaptability, efficiency, and feature enrichment. By using Gmapping, it was possible to generate maps from 2D LiDAR data, bypassing the constraints of 3D-specific algorithms like LeGO-LOAM. This allowed for a more practical solution when working with 2D sensors in real-world environments. Additionally, by limiting the data points to a radius around the robot, computational overhead was minimized without compromising the map's detail, ensuring that it remained sufficient for accurate localization. The inclusion of surface normals and curvature further enriched the map, enhancing its effectiveness for feature-based localization techniques. Ultimately, this approach ensured compatibility with 2D sensor data while providing a robust and reliable environmental representation, crucial for precise localization.

D. Optimal Particle Filter

The standard particle filter (PF) provides a robust framework for robot localization; however, its performance can degrade in environments with limited features, dynamic obstacles, or computational constraints. To address these limitations, an Optimal Particle Filter was employed, enhancing the traditional PF by optimizing the particle sampling and resampling strategies. Unlike the conventional approach, where particles

are uniformly sampled and weighted based on likelihood, the Optimal Particle Filter incorporates additional constraints and heuristics to focus sampling in areas of higher probability. This not only decreases the particle count needed for precise localization but also enhances computational efficiency.

The Optimal Particle Filter exploits the enriched feature set from the generated map, such as line normals and curvature by integrating them into the weight calculation. These features enable the filter to better align the particles with the map, ensuring more accurate pose estimates. The motion model remains the same as in the original approach by Andradi et al. [1], where the odometry motion model is approximated as a Gaussian distribution.

1) *Observation Model Adaptation for 2D Lines:* The observation model used in this work is primarily based on the closed-form Gaussian approximation approach proposed by Andradi et al. [1]. In the original formulation, surface normal information was leveraged from 3D LiDAR scan points and map point clouds to compute the observation likelihood. Specifically, the surface normals were used to identify point-to-plane correspondences, ensuring that only scan points with normals closely parallel to the ground plane were considered.

However, for this 2D case, where the input data consists of scan points represented as line segments, the original surface normal checks (e.g., ensuring alignment with the ground plane) are no longer applicable. Instead, modifications were made to adapt the observation model to work with 2D line features. The key change involves checking the orientation of the lines, ensuring that line correspondences are found based on angular alignment rather than surface normal alignment.

The modified observation model still follows the same Gaussian likelihood formulation for valid matches. Specifically, the likelihood of a match is computed as:

$$p(z_j|x_t) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(d_j \cdot n_j)^2}{2\sigma^2}\right)$$

Here: - d_j is the distance (or deviation) between the observed line segment and its corresponding line segment in the map. - n_j represents the orientation vector of the observed line.

To determine valid correspondences in 2D, angular deviations between line orientations are used instead of point-to-plane surface normals. This is computed as:

$$\cos^{-1}(\mathbf{n}_{z_j} \cdot \mathbf{n}_{m_j})$$

where $\mathbf{n}_{z_j}$ is the orientation vector of the observed line, and $\mathbf{n}_{m_j}$ is the orientation vector of the map line. Here, the angle between the two line orientation vectors is computed, ensuring correspondences are selected based on minimal angular misalignment. Line correspondences with high angular deviation or large distances are rejected as mismatches.

This adaptation is necessary because, in 2D, the concept of surface normals is replaced by line orientations. These modifications allow the observation model to efficiently handle 2D line-based data, preserving the core Gaussian likelihood formulation while adapting it to line features. All other steps, including downsampling the scan points, transforming the

observations to the map frame, and rejecting mismatches based on angular deviation and distance thresholds, remain consistent with the original approach described in [1].

2) *Optimal Particle Filter for 2D Localization*: For 2D localization, an optimal particle filter-based system is implemented using a Gaussian motion model and an adapted observation model. This filter follows three main steps—prediction, update, and resampling—at each time step, similar to the process described By Andradi et el [1]. However, the changes made to the observation model ensure compatibility with 2D data, specifically 2D line features, instead of 3D point clouds.

1) Prediction Step:

The proposal distribution is defined as:

$$q(x_t | x_{t-1}, Z_t, u_t) = \int p(Z_t | x_t) p(x_t | x_{t-1}, u_t) dx_t$$

The motion model $p(x_t | x_{t-1}, u_t)$ is modeled as a Gaussian distribution $\mathcal{N}(\mu_{\text{motion},t}, \Sigma_{\text{motion},t})$, and the observation model $p(Z_t | x_t)$ is also Gaussian: $\mathcal{N}(\mu_{\text{observation},t}, \Sigma_{\text{observation},t})$. The optimal proposal distribution for each particle is then computed as:

$$\begin{aligned} \mu_t^i &= \mu_{\text{motion},t} \Sigma_{\text{observation},t} + \\ &\mu_{\text{observation},t} \Sigma_{\text{motion},t} (\Sigma_{\text{motion},t} + \Sigma_{\text{observation},t})^{-1} \\ \Sigma_t^i &= (\Sigma_{\text{motion},t}^{-1} + \Sigma_{\text{observation},t}^{-1})^{-1} \end{aligned}$$

2) Update step:

The weight of each particle is computed as:

$$w_t^i = p(Z_t | \mu_t^i) p(\mu_t^i | x_{t-1}, u_t)$$

The likelihood of the observation, given the predicted particle pose, is computed using the Gaussian approximation:

$$A = (\mu_t^i - \mu_{\text{observation},t})^\top \Sigma_{\text{observation},t}^{-1} (\mu_t^i - \mu_{\text{observation},t})$$

$$p(Z_t | \mu_t^i) = \frac{1}{(2\pi)^{3/2} |\Sigma_{\text{observation},t}|^{1/2}} \cdot \exp\left(-\frac{1}{2}A\right)$$

In this step, instead of relying on surface normal alignment as in the 3D case, the correspondence between the observed and map lines is determined using angular alignment. The key difference is that the angular deviation between the orientations of the observed and map lines is used to assess the match:

$$\cos^{-1}(\mathbf{n}_{z_j} \cdot \mathbf{n}_{m_j})$$

Where $\mathbf{n}_{z_j}$ is the orientation vector of the observed line, and $\mathbf{n}_{m_j}$ is the orientation vector of the map line.

3) Resampling Step: The resampling step follows the same procedure as in the original implementation, where a low variance resampling technique is used. This ensures

that particles with high weights are retained while particles with low weights are discarded, thus maintaining computational efficiency.

This implementation of the Optimal Particle Filter for 2D localization is adapted from the original formulation by Andradi et al. [1], with modifications made to the observation model to accommodate 2D line-based features while preserving the core mathematical framework.

IV. EVALUATION

The evaluation of the proposed optimal particle filter algorithm was conducted by comparing it against the baseline particle filter, AMCL (Adaptive Monte Carlo Localization), which is a widely used ROS package for localization. The evaluation focused on two key aspects: robustness and accuracy.

For accuracy, the Root Mean Square Error (RMSE) was used as the primary metric. To assess robustness, the same experiments were repeated five times for each particle size to account for variability and consistency. Additionally, map discrepancies were introduced by modifying the map to include extra objects that were not present during the mapping phase.

The evaluations were performed in a simulation environment due to the unavailability of reliable ground truth data for real-world testing.

A. Software and Hardware Specifications

1) Software Specifications:

- Operating System: Ubuntu 20.04.6 LTS (Focal Fossa)
- ROS Version: ROS Noetic Ninjemys
- Python 3 Version: 3.8.10
- Python 2 Version: 2.7.18
- C++ Standard: C++14
- GCC Compiler Version: 9.4.0

2) Hardware Specifications:

- CPU: AMD Ryzen 9 (8-core/16-thread up to 5.2 GHz max boost)
- RAM: 16GB
- GPU: NVIDIA GeForce RTX 3050Ti, 4GB
- HDD: 1TB

B. Experimental Environments

In this study, the performance of the particle filter algorithms was evaluated in three distinct simulated environments. Each environment was designed to assess different challenges related to localization and map discrepancies.

1) *Simple Environment*: The first environment consisted of a basic layout with minimal obstacles, providing a straightforward path for the TurtleBot3. This setup aimed to establish a baseline for algorithm performance under ideal conditions, as shown in Figure 1.

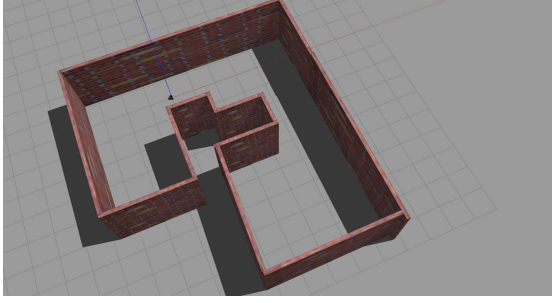


Fig. 1: Simple environment

2) *Circular Room*: This environment featured a large circular room with a 10m radius and a single opening leading to a corridor. To introduce map discrepancies, a large partition was added after mapping, creating ambiguity in the environment as shown in the figure 2.

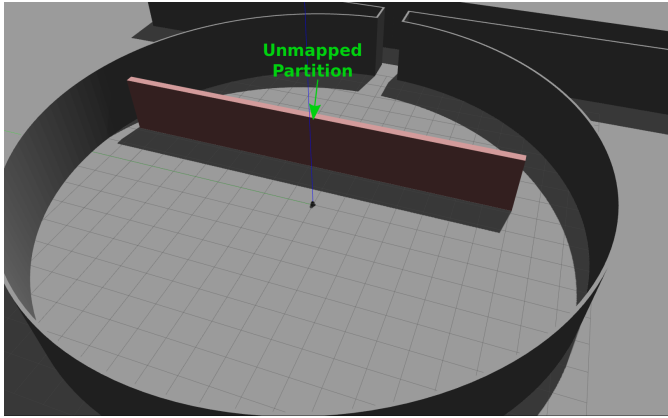


Fig. 2: Circular Room

3) *Similar Rooms*: The third environment comprised three large square rooms of 10m length, connected by a corridor. To simulate dynamic obstacles and map discrepancies, a closed door was added to the center room, which was absent during mapping as illustrated in the figure 3.

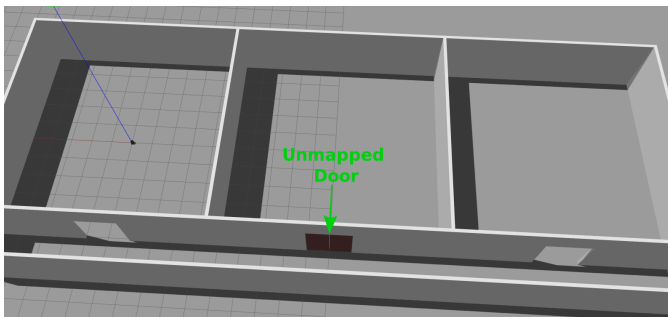


Fig. 3: Similar Rooms

C. Experimental Procedure

The experiments were conducted in a simulated environment to evaluate the performance of the proposed algorithm under controlled conditions. The primary objective was to analyze and compare the performance of particle filter algorithms,

specifically the Adaptive Monte Carlo Localization (AMCL) algorithm and the Optimal Particle Filter. These evaluations were carried out using a TurtleBot3 robot simulated in ROS Noetic, across the three distinct environments described earlier.

The AMCL algorithm was tested with its default parameters, as provided by the TurtleBot3 package, to serve as a baseline for comparison.

1) *Accuracy Test*: Accuracy in localization was evaluated by calculating the Root Mean Square Error (RMSE) between the estimated positions of the robot and the ground truth positions. RMSE provides a quantitative measure of the algorithm's performance, with lower values indicating higher accuracy.

To ensure a comprehensive evaluation, RMSE was calculated at each time step during the robot's operation. This allowed for a detailed analysis of the algorithm's ability to consistently track the robot's pose over time, even as it moved through different environments and encountered challenges such as map discrepancies and dynamic obstacles.

2) *Robustness Test*: Robustness testing of the particle filter algorithms was conducted in three distinct environments to evaluate their performance under challenging conditions. The primary goal was to assess the algorithms' ability to maintain accurate localization despite ambiguities, map discrepancies, and dynamic changes in the environment.

The robustness of the particle filter algorithms was evaluated under challenging conditions to assess their ability to maintain accurate localization despite ambiguities and map discrepancies. A successful run was defined as one where the robot's final pose was within a 0.5-meter radius of the ground truth pose. Ambiguity was tested in environments designed to confuse the localization system, such as the identical rooms and circular room scenarios, where features like the closed door and unmapped partition introduced uncertainty. Robustness to map discrepancies was analyzed by evaluating the algorithms' performance when the environment differed from the map, as described earlier. The accuracy and robustness results were visualized using line graphs and box plots: line graphs highlighted RMSE trends across runs, with lighter colors representing individual runs and darker colors showing the median of five runs, while box plots provided a clear view of RMSE variability across runs, enabling an insightful comparison of algorithm performance.

- *Direct Motion Accuracy Test*

The first scenario involved conducting three motion accuracy tests under straightforward conditions. The robot was spawned at the origin (0, 0), and the tests began without any initial perturbations or challenges. These tests aimed to measure the RMSE between the Odometry (Odom) position and the filter's estimated position during different types of motion. These experiments were repeated for particle quantities of 1, 3, 50, 100, 250, and 500, with each configuration being run five times to ensure robustness and consistency in the results. This test was performed in the simple room.

- 1) *Linear Motion Accuracy Test*

- The TurtleBot3 moved linearly along the x-axis

at a velocity of 0.22 m/s, covering a distance of approximately 5 meters, as shown in fig 4.

- Both the Odom position and the filter position started at (0, 0) before motion.
- The robot's trajectory followed a straight line along the x-axis.
- RMSE data were collected for each run, with an average value calculated at the end of the entire run.

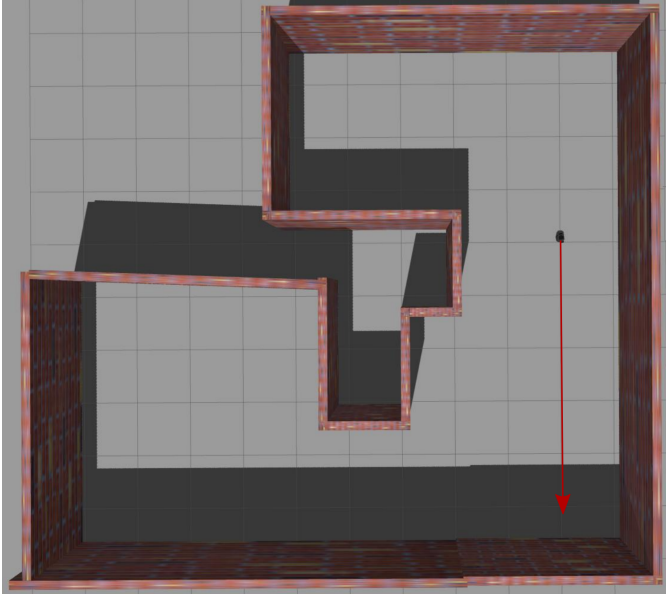


Fig. 4: Simple room linear motion

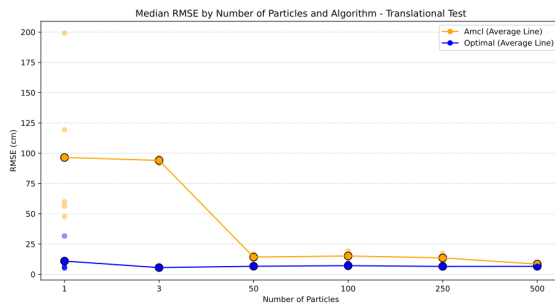


Fig. 5: Median RMSE - Translational Test

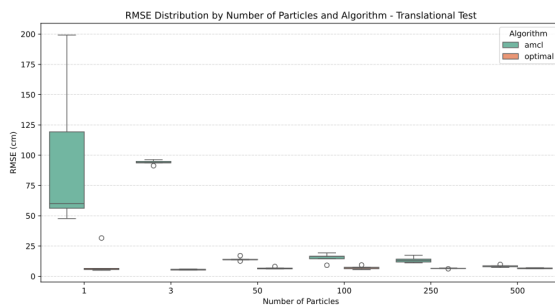


Fig. 6: Box Plot - Translational Test

2) Rotational Motion Accuracy Test

- The TurtleBot3 executed a full circular motion with a constant linear velocity of 0.22 m/s and an angular velocity of -0.5 rad/s, as shown in fig 7.
- Both the Odom position and the filter position started at (0, 0) before motion.
- The robot's trajectory described a perfect circle.
- RMSE data were collected for each run, with an average value calculated at the end of the entire run.

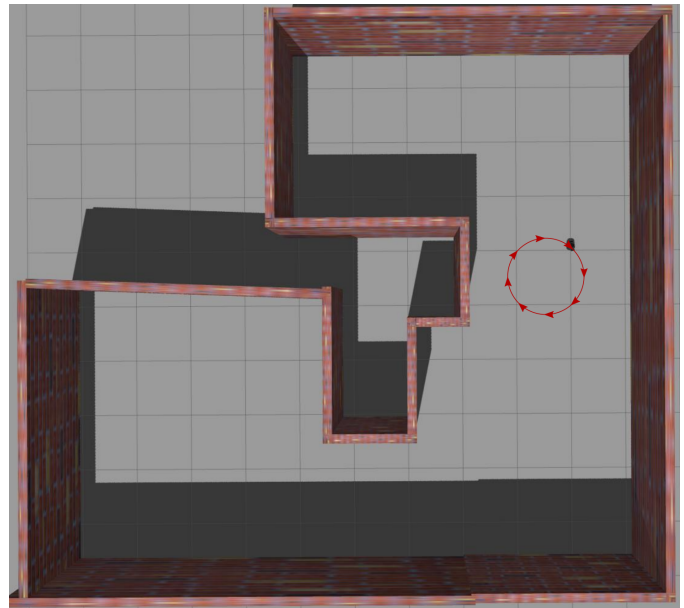


Fig. 7: Simple room rotational motion

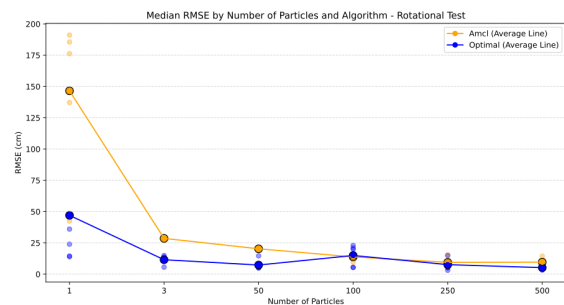


Fig. 8: Median RMSE - Rotational Test

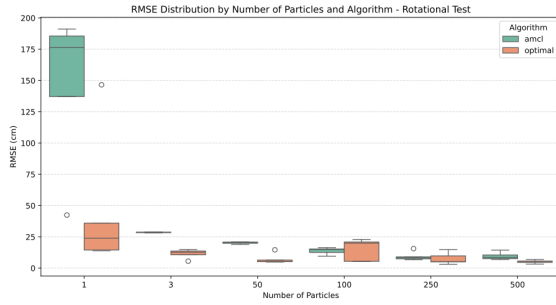


Fig. 9: Box Plot - Rotational Test

3) Random Motion Accuracy and Robustness Test

- The TurtleBot3 underwent random motion involving a combination of translational and rotational velocities dynamically assigned via a predefined Python script.
- Both the Odom position and the filter position started at (0, 0) before motion.
- The robot's trajectory was a mix of translation and rotation.
- RMSE data were collected for each run, with an average value calculated at the end of the entire run.

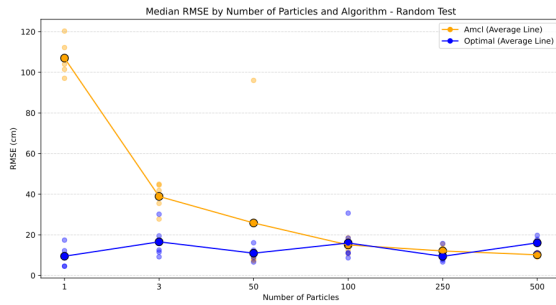


Fig. 10: Median RMSE - Random Test

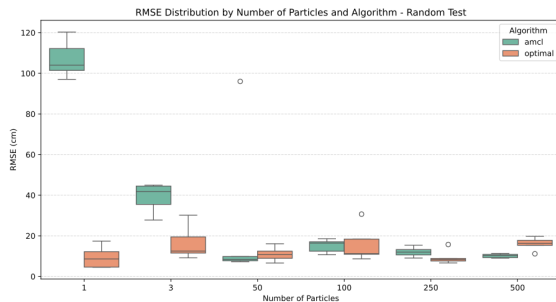


Fig. 11: Box Plot - Random Test

• Robot Kidnap Scenario Tests

The second scenario involved simulating a "robot kidnap" problem to evaluate the robustness and adaptability of the particle filter algorithms. In this experiment, the TurtleBot3 was manually displaced to random positions

after spawning by lifting and moving it along the linear x and y axes, as well as performing rotational adjustments. This dynamic setup aimed to emulate real-world situations where the robot might be physically picked up and placed at a random position, creating significant uncertainty in localization. Following each manual displacement, the TurtleBot3 was subjected to a random motion accuracy test to assess the algorithms' ability to recover and maintain accurate localization. A figure illustrating the displacement locations is shown in Fig. 12 to clarify this process, and the corresponding plots for these experiments are also included to visualize the results.

As in the previous scenario, these experiments were repeated for particle quantities of 1, 3, 50, 100, 250, and 500, with each configuration run five times to ensure consistency and reliability in the results.

For one of the tests, all the data from the run were plotted without averaging, offering a comprehensive examination of the algorithms' performance across varying particle quantities. This detailed visualization provides insight into how both algorithms recover after the robot is displaced. The plotted results emphasize the effective recovery capabilities of the Optimal Particle Filter in the robot kidnap scenario, in contrast to AMCL, which exhibits difficulties in regaining accurate localization under these challenging conditions.

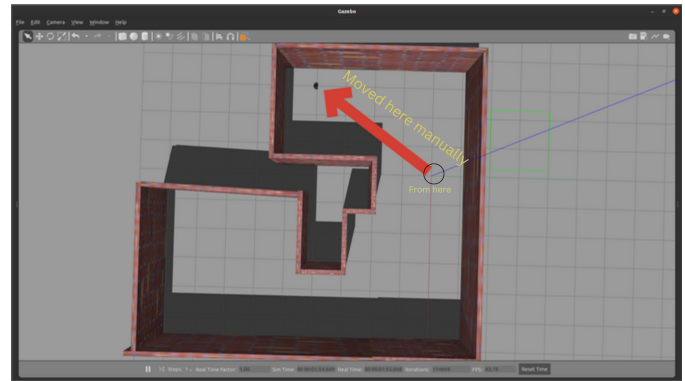


Fig. 12: Robot Kidnap

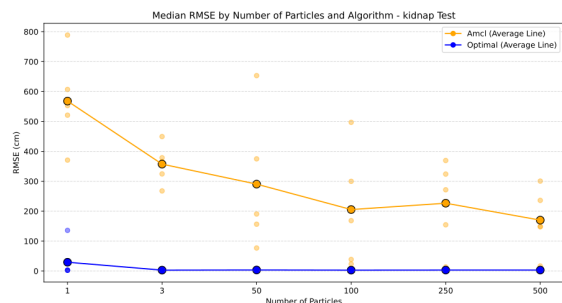


Fig. 13: Median RMSE - Kidnap Test

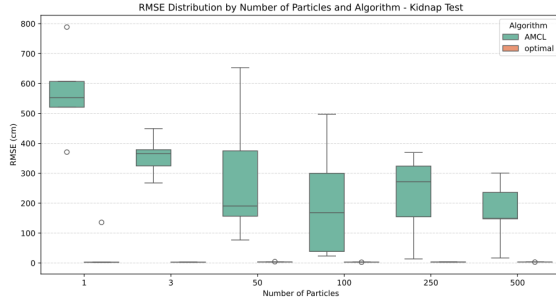


Fig. 14: Box Plot - Kidnap Test

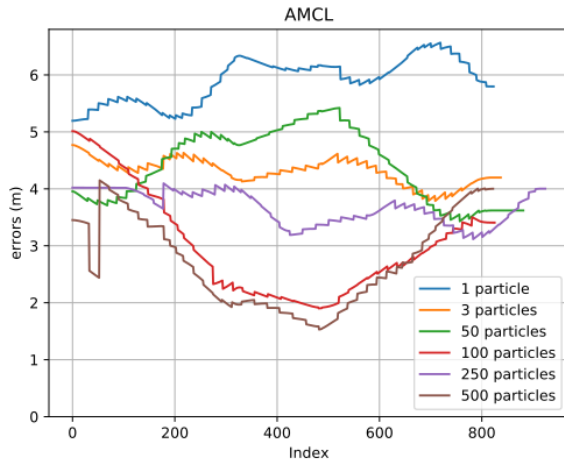


Fig. 15: AMCL Errors for Different Particle Quantities

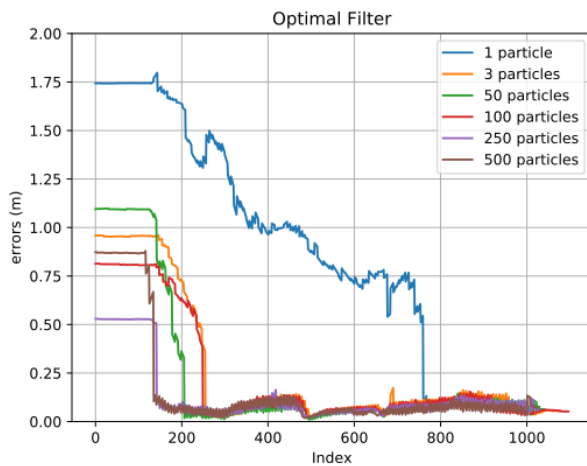


Fig. 16: Optimal Filter Errors for Different Particle Quantities

- *Random Wrong Initial Pose Test*

In this experiment, the TurtleBot3 robot was initially spawned at (0,0), but its starting pose was deliberately altered within a 1-meter radius of the origin using RViz 2D Pose Estimation. This was performed under only random motion conditions to evaluate how well the algorithms could handle inaccurate starting positions.

The tests were conducted for particle quantities of 1, 3, 50, 100, 250, and 500, with each configuration run 5 times to ensure robust results. The evaluation focused on both accuracy and robustness, with RMSE visualized through line graphs and the spread of RMSE values across runs depicted using box plots.

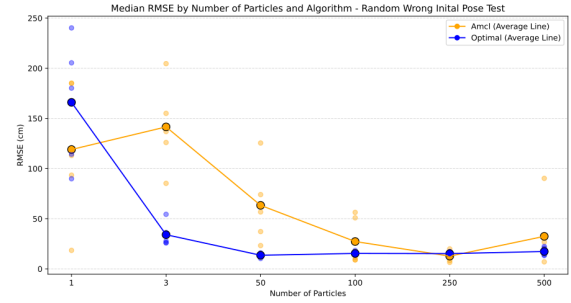


Fig. 17: Median RMSE - Random Wrong Initial Pose Test

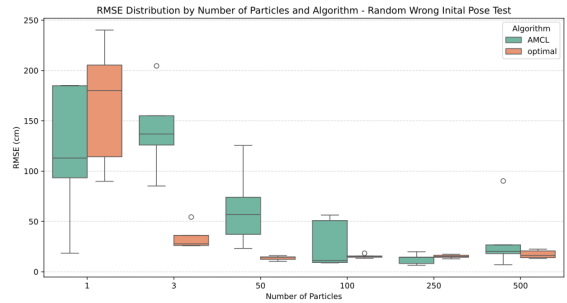


Fig. 18: Box Plot -Random Wrong Initial Pose Test

- *Three Identical Rooms*

This experiment was conducted in the similar rooms environment, consisting of three identical rooms with closed doors. The identical features across the rooms posed a significant challenge for the particle filter algorithms, as the lack of unique characteristics made accurate localization more difficult. The path taken by the robot during this test is as shown in Fig 19. To evaluate performance, line graphs 20 were plotted to show the RMSE for both AMCL and the Optimal Particle Filter in this environment, and a box plot 21 was used to visualize the spread of RMSE values across the runs. Additionally, a table I was created to summarize the successful runs for each particle quantity (1, 3, 50, 100, 250, and 500) for both algorithms. Each particle quantity was tested over 5 runs, with a value of 0 in the table indicating a failure to localize successfully by the end of the run and a value of 1 indicating successful localization, with the robot's pose within a 0.5-meter radius of the true pose at the end of the run.

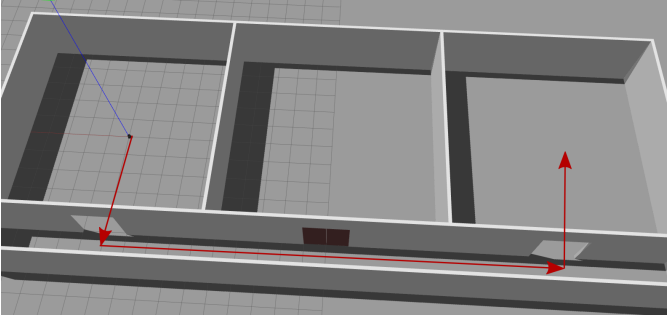


Fig. 19: Three identical square rooms of 10m length environment

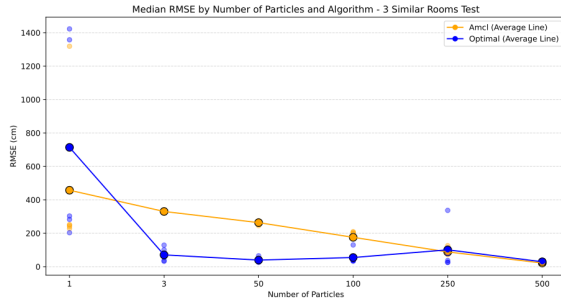


Fig. 20: Median RMSE - Similar Rooms Test

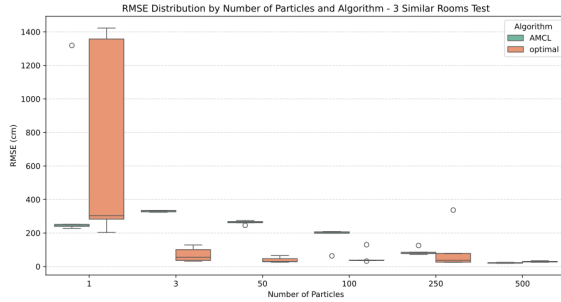


Fig. 21: Box Plot - Similar Rooms Test

Number of particles	AMCL					Optimal				
	1	2	3	4	5	1	2	3	4	5
1	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	1	1	1	1	1
50	0	0	0	0	0	1	1	1	1	1
100	0	0	0	0	0	1	1	1	1	1
250	1	1	1	1	1	1	1	1	1	1
500	1	1	1	1	1	1	1	1	1	1

TABLE I: AMCL and Optimal Successful Runs in Similar Rooms

- *Circular Room*

This experiment was conducted in the circular room environment, a 20-meter diameter space that presented

challenges for the particle filter algorithms due to the lack of unique features. The path taken by the robot during this test is as shown in Fig 22. Similar to the previous experiment in the similar rooms environment, line graphs 23 were plotted to show the RMSE for both AMCL and the Optimal Particle Filter, with a box plot 24 visualizing the spread of RMSE values across the runs. A table was also created to summarize the successful runs for each particle quantity (1, 3, 50, 100, 250, and 500), with 5 runs conducted for each configuration. In the table, a value of 0 indicates that the robot failed to localize successfully by the end of the run, while a value of 1 signifies successful localization, with the robot's pose within a 0.5-meter radius of the true pose at the end of the run.

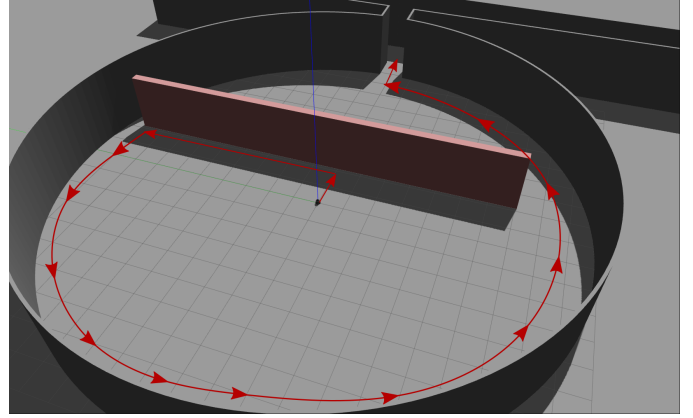


Fig. 22: Circular room of 20m diameter environment

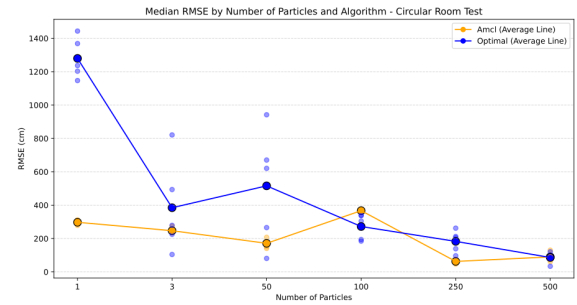


Fig. 23: Median RMSE - Circular Rooms Test

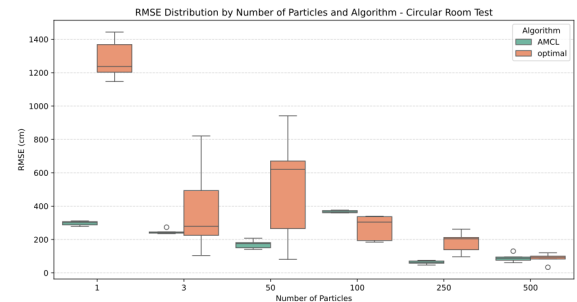


Fig. 24: Box Plot - Circular Rooms Test

Number of particles	AMCL					Optimal				
	1	2	3	4	5	1	2	3	4	5
1	0	0	0	0	0	0	0	0	0	0
3	0	0	0	0	0	0	0	0	0	0
50	0	0	0	0	0	1	0	0	0	0
100	0	0	0	0	0	0	1	0	0	0
250	0	0	0	0	0	0	0	1	1	1
500	0	0	0	0	0	1	1	1	0	0

TABLE II: AMCL and Optimal Successful Runs in Circular Rooms

3) *Runtime Test* : Two aspects of runtime were evaluated to analyze the performance of the particle filter algorithms.

First, the time taken to localize the robot was measured for both the Kidnapped Pose III IV and Random Wrong Initial Pose V VI tests, comparing the AMCL and Optimal Particle Filter algorithms. A table is provided summarizing the localization times (in seconds) for varying particle quantities (1, 3, 50, 100, 250, and 500), with five runs conducted for each configuration. Cases where the robot failed to localize successfully within the run are represented as NaN values.

No. of particles	1	2	3	4	5
1	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
50	NaN	NaN	NaN	NaN	1.867
100	NaN	NaN	NaN	NaN	NaN
250	NaN	NaN	NaN	2.066	NaN
500	NaN	NaN	NaN	43.034	2.733

TABLE III: AMCL Time Taken to Localize (seconds) - Robot Kidnap Test

No. of particles	1	2	3	4	5
1	0	0	0	0	0
3	0	0	0	0	0
50	0	0	0	0	0
100	0	0	0	0	15.338
250	0	0	0	6.8	0
500	0	0	0	0	0.02

TABLE IV: Optimal Time Taken to Localize (seconds) - Robot Kidnap Test

No. of particles	1	2	3	4	5
1	NaN	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN	NaN
50	NaN	NaN	NaN	NaN	NaN
100	NaN	3.035	5.101	3.201	NaN
250	9.7	13.667	2.17	1.303	1.434
500	NaN	7.4	11.166	4.9	4

TABLE V: AMCL Time Taken to Localize (seconds) - Random Wrong Initial Pose Test

No. of particles	1	2	3	4	5
1	NaN	NaN	NaN	NaN	NaN
3	1.173	18.816	0	24.174	8.386
50	6.169	10.732	0	2.336	10.987
100	4.71	1.597	22.367	23.267	5.474
250	27.654	9.845	18.463	29.124	0
500	0.5	5.933	32	30.834	2.07

TABLE VI: Optimal Time Taken to Localize (seconds) - Random Wrong Initial Pose Test

Second, the computational efficiency of the Optimal Particle Filter was assessed by measuring the time taken (in milliseconds) to process each particle under different particle quantities. This data was collected exclusively for the Optimal Particle Filter since direct access to the AMCL code was unavailable, preventing similar measurements for the AMCL algorithm. This test was conducted in two environments: the three similar rooms 25 and the circular room 26, to evaluate performance consistency across different challenging scenarios.

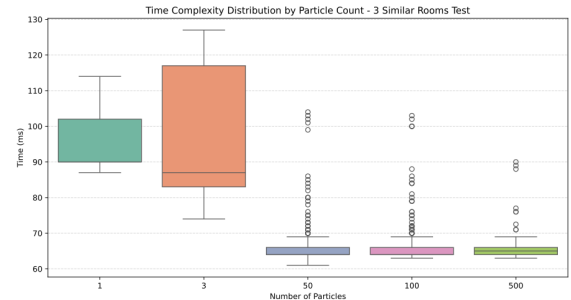


Fig. 25: Time Complexity Distribution by Particle Count - Similar Rooms Test

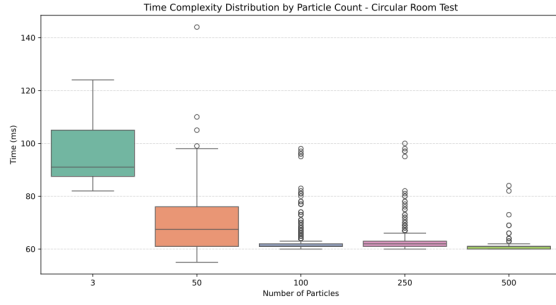


Fig. 26: Time Complexity Distribution by Particle Count - Circular Room Test

V. CONCLUSIONS

A. Summary

In this project, an efficient and robust 2D LiDAR-based indoor localization system was developed for wheeled mobile robots with noisy odometry. Building upon prior work in 3D environments [1], the system employs optimized particle filter techniques to achieve high accuracy and robustness. A high-fidelity observation model was used for particle propagation, while a lower-fidelity observation model handled weight updates, ensuring accuracy without compromising robustness.

The proposed system was evaluated in both simulated and real-world scenarios, effectively addressing challenges such as ambiguity and map discrepancies. Comparative analysis with standard particle filter-based localization systems, like AMCL, revealed significant improvements in performance, accuracy, and efficiency. The system demonstrated the feasibility of achieving superior localization accuracy with a reduced number of particles, while maintaining manageable computational overhead. These findings highlight the system's robustness in handling challenging scenarios, such as ambiguous initial poses, while offering a more efficient and reliable alternative to traditional localization methods.

In various motion tests (direct, translational, rotational, and random), the proposed algorithm consistently outperformed AMCL, even with fewer particles (1 or 3), yielding better results than AMCL with 250 particles. This highlights the efficiency of the optimal particle filter in achieving more accurate localization.

In the robot kidnapping test, where the robot was manually repositioned, AMCL struggled to recover effectively, even with a particle count of 500. In contrast, the optimal particle filter demonstrated better adaptability and localization performance, even with a reduced number of particles, as evidenced by the error plots. This showcases the superior ability of the optimal particle filter to handle complex and dynamic situations, making it more efficient and reliable than AMCL in such scenarios.

Additionally, in cases of random wrong initial poses and similar room configurations, the proposed approach effectively recovered and localized even with 3 particles, outperforming AMCL in both accuracy and robustness. However, in the circular room test, the proposed algorithm showed reduced

performance, with very few successful localization runs, while AMCL was unable to localize at all.

These findings demonstrate the robustness and efficiency of the proposed algorithm in various scenarios, with notable improvements in accuracy and reliability compared to AMCL, even with fewer particles. The results highlight the effectiveness of the proposed approach in handling localization challenges, particularly in dynamic and ambiguous environments.

B. Contributions

The primary contribution of this project lies in the development and evaluation of an efficient and robust 2D LiDAR-based indoor localization system, which successfully integrates optimized particle filter techniques to address challenges faced by traditional localization methods. The system offers improved accuracy, adaptability, and robustness in dynamic environments, especially with noisy odometry data. One of the key innovations of the proposed approach is its ability to achieve high localization performance with fewer particles, making it computationally efficient while maintaining high accuracy.

A notable contribution is the adaptation of a previously established 3D LiDAR-based localization [1] approach to 2D environments. By leveraging optimized particle filter techniques and incorporating surface normals, the system demonstrates effectiveness and versatility across various challenging scenarios. This adaptation not only addresses traditional limitations in 2D localization but also brings innovations from 3D localization techniques, enabling better handling of real-world challenges such as noisy odometry, dynamic obstacles, and ambiguous initial poses. It significantly expands the applicability of existing 3D methods to 2D, pushing the boundaries of localization techniques for mobile robots.

Another key contribution is the enhancement of particle filter-based approaches in the field of mobile robot localization. By showing that accurate localization can be achieved with as few as one or three particles, the system demonstrates significant improvements over traditional methods, such as AMCL, especially in environments with ambiguous initial poses and challenging conditions. This ability to function efficiently with fewer particles has the potential to optimize localization algorithms for real-time, resource-constrained applications.

Additionally, the system's performance in robot kidnapping scenarios, where the robot is manually repositioned demonstrates the robustness of the approach in handling dynamic and unpredictable environments. This showcases the system's adaptability to complex and changing environments without requiring a large number of particles, a crucial advancement in localization research.

Another important contribution is the comprehensive evaluation of the proposed algorithm across various motion tests and environments. These experiments provide valuable insights into its strengths and weaknesses, highlighting the system's potential for real-world applications, while also identifying areas for future improvement, such as more efficient map generation techniques and better handling of non-planar environments.

Overall, this work provides a robust solution to the problem of 2D indoor localization for mobile robots, pushing the boundaries of particle filter algorithms by demonstrating their applicability in real-time, resource-constrained environments.

C. Limitations and Future Work

The proposed approach in this project faces several limitations that could be addressed in future work. One of the primary challenges is the reliance on the GMapping algorithm for map generation, which involves a time-consuming and computationally expensive process of converting LiDAR scans into point clouds and then extracting surface normals. This approach may not be efficient for larger environments or real-time applications. Additionally, similar to the 3D surface normal-based approach [1], the proposed 2D system relies on finding planar structures, which may not always be available or sufficiently distinctive in certain environments. Environments with sparse features or cluttered spaces could lead to poor localization performance. Another limitation is the use of a fixed particle size, which may hinder the system's adaptability to varying conditions such as environmental changes or dynamic obstacles. While the system is robust against noisy odometry, sensor noise remains an issue, especially in environments with reflective or irregular surfaces, which can affect the quality of LiDAR scans and impact the accuracy of localization.

For future work, one promising direction is to improve the map generation process and the normal extraction technique. By developing more efficient mapping algorithms or utilizing direct LiDAR-to-point-cloud conversion methods, the system could reduce the time and computational resources required for localization, leading to significantly improved performance. Additionally, implementing an adaptive particle filter would allow the system to adjust the number of particles based on the uncertainty in the robot's pose, improving both efficiency and adaptability. This would allow the system to use fewer particles in stable environments and increase the number of particles in situations with higher uncertainty or more complex dynamics. Further exploration could involve handling non-planar environments more effectively, possibly through the integration of techniques like Non-Uniformity-based methods or NDT (Normal Distributions Transform) maps, which could enhance the system's ability to localize in environments with irregular or sparse features.

The system could also be extended to handle faster robots or those with lower scan rates. Moreover, the algorithm has not been thoroughly tested in real-world scenarios with reliable ground truth, limiting the ability to fully validate its performance outside of simulation. Future work should focus on conducting experiments in real environments with access to ground truth data to better assess the system's accuracy and robustness in practical applications. Further research is also needed to ensure that the approach can maintain accuracy and efficiency at higher speeds or with reduced scan frequencies. Finally, sensor fusion could play a critical role in enhancing the robustness of the system. Incorporating additional sensors such as cameras, IMUs, or proprioceptive sensors could provide complementary information, improving localization

accuracy, particularly in environments with dynamic obstacles or varying lighting conditions.

By addressing these limitations and following the outlined future directions, the system could be made more efficient, adaptable, and robust across a broader range of environments and real-world applications.

REFERENCES

- [1] H. Andradi, S. Blumenthal, E. Prassler, and P. G. Plöger, "Lidar-based indoor localization with optimal particle filters using surface normal constraints," in *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 2023, pp. 1947–1953.
- [2] F. Dellaert, D. Fox, W. Burgard, and S. Thrun, "Monte carlo localization for mobile robots," in *Proceedings 1999 IEEE International Conference on Robotics and Automation (Cat. No.99CH36288C)*, vol. 2, 1999, pp. 1322–1328 vol.2.
- [3] D. Fox and W. Burgard, "Markov localization for mobile robots in dynamic environments," *J. Artif. Intell. Res.*, vol. 11, 12 1999.
- [4] A. Q. Li, M. Xanthidis, J. M. O'Kane, and I. Rekleitis, "Active localization with dynamic obstacles," in *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2016, pp. 1902–1909.
- [5] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34–46, 2007.
- [6] J. L. Blanco, J. González-Jiménez, and J.-A. Fernández-Madriral, "Optimal filtering for non-parametric observation models: Applications to localization and slam," *I. J. Robotic Res.*, vol. 29, pp. 1726–1742, 12 2010.
- [7] S. Jianchao, Z. Xuebo, S. Lei, and L. Jingtai, "Map-based robust localization for indoor mobile robots," in *2017 36th Chinese Control Conference (CCC)*, 2017, pp. 6945–6950.
- [8] D. Meyer-Delius, J. Hess, G. Grisetti, and W. Burgard, "Temporary maps for robust localization in semi-static environments," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2010, pp. 5750–5755.
- [9] R. Valencia, J. Saarinen, H. Andreasson, J. Vallvé, J. Andrade Cetto, and A. Lilienthal, "Localization in highly dynamic environments using dual-timescale ndt-mcl," 05 2014.
- [10] G. D. Tipaldi, D. Meyer-Delius, and W. Burgard, "Lifelong localization in changing environments," *The International Journal of Robotics Research*, vol. 32, pp. 1662 – 1678, 2013. [Online]. Available: <https://api.semanticscholar.org/CorpusID:2462733>
- [11] S. Thrun, "Probabilistic robotics," *Communications of the ACM*, vol. 45, no. 3, pp. 52–57, 2002.
- [12] ROS-Perception, "laser_geometry package," http://wiki.ros.org/laser_geometry.

ACKNOWLEDGMENT

I would like to express my sincere gratitude to my supervisors, Prof. Dr. Paul G. Plöger and Deebul Nair, for their invaluable guidance, support, and encouragement throughout the course of this project. Their expert advice, insightful feedback, and constant availability were instrumental in shaping the direction of this research. I would also like to extend my heartfelt thanks to my colleagues for their constructive discussions and assistance, which significantly contributed to the completion of this work.

Special thanks go to Hochschule Bonn-Rhein-Sieg and KELO Robotics GmbH for providing the necessary resources and equipment that made this project possible. I am also grateful to my family and dear friends for their constant support and encouragement throughout my studies.

STATEMENT OF ORIGINALITY

I, the undersigned below, declare that this work has not previously been submitted to this or any other university and that it is, unless otherwise stated, entirely my own work. The report was, in part, written with the help of the AI assistant ChatGPT as described in the appendix. I am aware that content generated by AI systems is no substitute for careful scientific work, which is why all AI-generated content has been critically reviewed by me, and I take full responsibility for it.

05.01.2025

Date

M.D. Waagem

Signature

APPENDIX

D. Additional Evaluation Results

In addition to the main evaluation presented in the report, additional tests were conducted to compare the performance of the 2D localization algorithm under different conditions. The tests included scenarios involving linear, rotational, and random motion, as well as a robot kidnap scenario in simple room environment. These tests were performed using the 2D localization system with interpolated data, as well as the direct line-based method and AMCL, for comparison.

However, it should be noted that only a single run was performed for each particle size in these tests, which limits the statistical significance of the results. Multiple runs for each particle size were not conducted due to time constraints. As such, these results should be interpreted as preliminary findings and not as definitive evaluations of the system's performance.

The results are presented in the following plots that compare the performance of the 2D system with interpolated data, the 2D system using direct line-based data, and AMCL. These plots provide insight into the behavior of the algorithms under different motion types and scenarios, but further experimentation with multiple runs per particle size is necessary to obtain more robust conclusions.

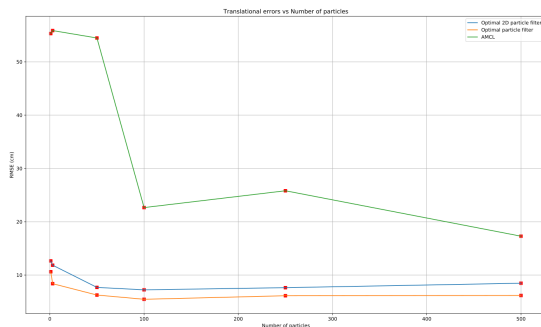


Fig. 27: Translational Motion Accuracy – Direct Method

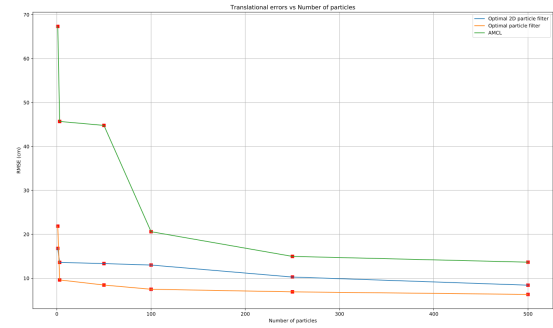


Fig. 28: Rotational Motion Accuracy – Direct Method

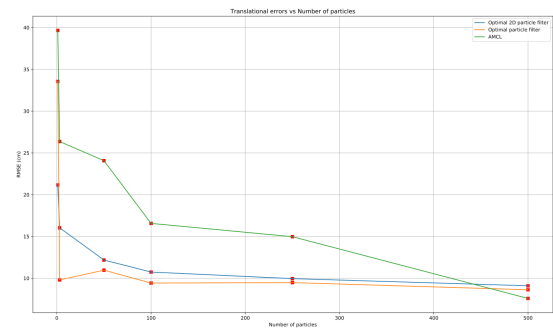


Fig. 29: Random Motion Accuracy – Direct Method

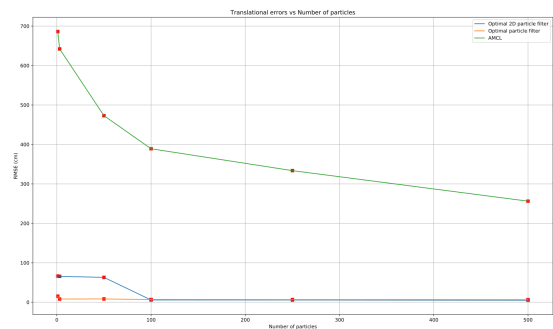


Fig. 30: Translational Motion Accuracy – Robot Kidnap Method

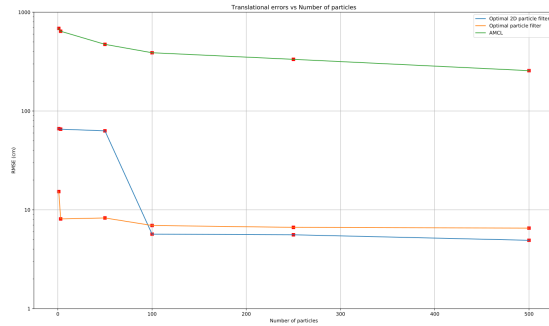


Fig. 31: Translationa Motion Accuracy – Robot Kidnap Method LOG

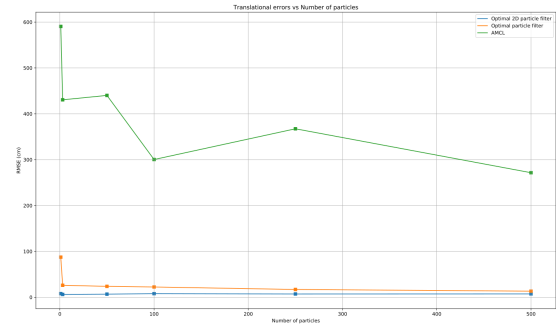


Fig. 34: Random Motion Accuracy – Robot Kidnap Method LOG

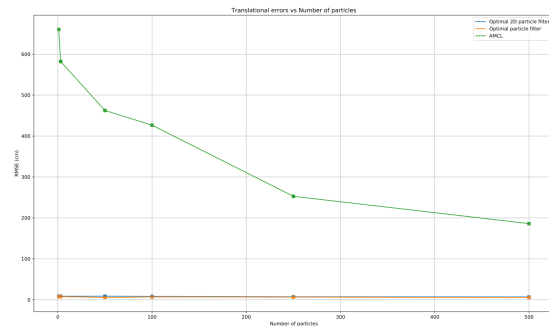


Fig. 32: Rotational Motion Accuracy – Robot Kidnap Method LOG

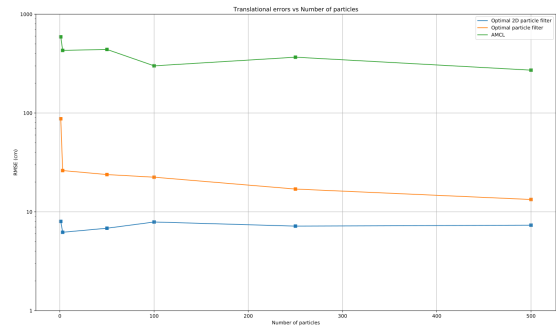


Fig. 35: Random Motion Accuracy – Robot Kidnap Method LOG

E. Use of AI Assistance

In the course of writing this report, I utilized the AI assistant ChatGPT to assist with content generation, refinement, and proofreading. The AI was primarily used to help draft and structure various sections of the report, improving clarity, flow, and language of technical descriptions and complex ideas, as well as suggesting alternative formulations. Additionally, ChatGPT supported the refinement of the document by offering suggestions for grammar, sentence structure, and overall readability. All AI-generated content was critically reviewed, edited, and adapted by me to ensure that it met the standards of accuracy, relevance, and alignment with the objectives of the report. While AI assistance contributed to the writing process, I take full responsibility for the final content presented in this report.

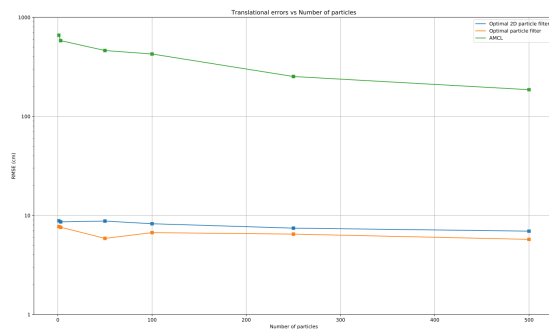


Fig. 33: Rotational Motion Accuracy – Robot Kidnap Method LOG